

GPUMODE Lecture Study Notes: Lecture 8: CUDA Performance Checklist

Contents

1 Core Problem the Lecture Addresses

The lecture addresses the practical problem of making CUDA kernels fast by turning profiling observations into concrete optimization actions. The central theme is that a GPU is worth using only when it delivers performance, and CUDA performance is usually controlled by a small set of recurring bottlenecks.

The speaker frames the lecture as a direct sequel to an earlier profiling lecture. The workflow is:

1. Write a CUDA kernel.
2. Form a hypothesis about the bottleneck.
3. Compile and run the kernel.
4. Profile it with NVIDIA Nsight Compute, commonly invoked as `ncu`.
5. Inspect metrics such as DRAM throughput, L1 cache throughput, occupancy, warp statistics, and kernel duration.
6. Modify the kernel using a known optimization pattern.
7. Re-profile to validate whether the change helped.

The lecture's performance checklist consists of the following main ideas:

- Coalesce global memory accesses.
- Maximize occupancy without blindly maximizing thread count.
- Understand whether the kernel is memory-bound or compute-bound.
- Reduce control-flow divergence inside warps.
- Tile reused data into faster memory.
- Privatize intermediate data into registers or shared memory.
- Use thread coarsening when memory traffic dominates.
- Rewrite algorithms using better mathematical identities when possible.

A CUDA performance checklist is not a bag of random tricks. It is a decision tree: profile first, identify whether the bottleneck is memory, compute, occupancy, scheduling, or divergence, and then choose the optimization that targets that bottleneck.

2 Required Background

2.1 GPU Memory Hierarchy

A CUDA kernel interacts with several levels of memory. The lecture emphasizes that most performance optimizations are attempts to reduce expensive global memory traffic and use faster memory more effectively.

- **Global memory / DRAM:** large, off-chip, high bandwidth, high latency. It can store tens of gigabytes on modern GPUs.
- **L2 cache:** shared across SMs. Faster than DRAM but still not under explicit programmer control.
- **L1 cache:** closer to each SM. Lower latency than L2 and DRAM, but still not directly controlled like shared memory.
- **Shared memory / SRAM:** explicitly managed on-chip memory within an SM. Much smaller than global memory but much faster.
- **Registers:** private per-thread storage. Fastest, but limited. Excessive register usage can reduce occupancy.

The physical reason SRAM is scarce is that it is more expensive in area and power than DRAM. DRAM can be implemented with roughly one transistor and one capacitor per bit, whereas SRAM commonly uses about six transistors per bit. This makes SRAM faster but physically larger, hotter, and more expensive.

2.2 Latency, Bandwidth, and Throughput-Oriented Design

GPUs are throughput-oriented machines. They do not usually reduce memory latency directly; instead, they hide latency by scheduling many warps and issuing other work while some warps wait for memory.

A useful latency mental model is:

More parallel lanes can increase throughput, but they do not automatically reduce latency.

If one memory request takes many cycles, launching enough independent warps can hide the delay, but each individual memory transaction is still slow. This is why occupancy, coalescing, and memory reuse are so important.

2.3 CUDA Execution Model

CUDA programs launch grids of thread blocks. Each block contains threads, and the hardware executes threads in groups of 32 called warps.

The standard one-dimensional indexing formula is:

$$i = \text{blockIdx.x} \cdot \text{blockDim.x} + \text{threadIdx.x}.$$

Here:

- `blockIdx.x` identifies which block the thread belongs to.
- `blockDim.x` is the number of threads in a block.
- `threadIdx.x` identifies the thread within the block.
- i is the global linear index into a flat memory buffer.

Even if tensors are conceptually multi-dimensional, the memory backing them is usually linear. Multi-dimensional indexing is lowered into address arithmetic over this flat buffer.

2.4 Profiling Tools

The lecture uses Nsight Compute, commonly invoked as:

```
ncu ./benchmark
```

The relevant metrics include:

- Kernel duration.
- DRAM throughput.
- L1 cache throughput.
- Memory workload warnings.
- Theoretical occupancy.
- Achieved occupancy.
- Warp state statistics.
- Branch instruction counts.
- Branch efficiency and divergent branch metrics.

Because CUDA kernels launch asynchronously, CPU-side timing must include synchronization:

```
cudaDeviceSynchronize();
```

Without synchronization, the host may record a time before the GPU has actually finished the kernel.

3 Main Concepts

3.1 Coalescing Global Memory Accesses

Coalescing means arranging memory accesses so that adjacent threads in a warp read or write adjacent memory locations. A warp has 32 threads, and hardware memory transactions are efficient when those threads access contiguous addresses.

A coalesced copy kernel looks like:

```
__global__ void copy_data_coalesced(float* out, const float* in, int n) {  
    int index = blockIdx.x * blockDim.x + threadIdx.x;  
  
    if (index < n) {  
        out[index] = in[index];  
    }  
}
```

Each thread t reads $\text{in}[t]$ and writes $\text{out}[t]$. Within a warp, consecutive threads access consecutive memory locations:

thread 0 $\rightarrow$ $\text{in}[0]$, thread 1 $\rightarrow$ $\text{in}[1]$, ..., thread 31 $\rightarrow$ $\text{in}[31]$.

A deliberately non-coalesced variant may use a strided or permuted index:

```

__global__ void copy_data_noncoalesced(float* out, const float* in, int n) {
    int index = blockIdx.x * blockDim.x + threadIdx.x;

    if (index < n) {
        int read_index = (2 * index) % n;
        out[index] = in[read_index];
    }
}

```

This breaks the simple mapping between adjacent threads and adjacent addresses. The warp now touches a less contiguous memory pattern, which can increase memory transactions, reduce cache usefulness, and increase kernel duration.

The lecture reports that the non-coalesced version had higher DRAM pressure and lower L1 cache effectiveness. The coalesced version reduced total runtime substantially, with a duration around $558\mu s$ compared with about $764\mu s$ for the non-coalesced version in the demonstrated setup.

Visual Concept

Draw a warp of 32 threads on the left and a row of global memory addresses on the right. In the coalesced case, draw straight parallel arrows from thread i to address i . In the non-coalesced case, draw crossing arrows that skip or wrap around memory locations.

3.2 Occupancy

Occupancy measures how many warps are active on an SM relative to the maximum number of warps the SM can support.

A simplified expression is:

$$\text{Occupancy} = \frac{\text{active warps per SM}}{\text{maximum resident warps per SM}}.$$

High occupancy helps hide latency because when one warp stalls on memory, the scheduler can issue instructions from another ready warp.

However, occupancy is constrained by:

- Threads per block.
- Registers per thread.
- Shared memory per block.
- Maximum blocks per SM.
- Maximum warps per SM.
- Hardware generation.

The lecture shows that simply changing the block size can change occupancy. For example, increasing a block size from 128 to 1024 improved a reported theoretical occupancy from roughly 77% to roughly 86% in one experiment.

3.3 Tile Quantization and Wave Quantization

Occupancy issues can appear through two related effects.

3.3.1 Tile Quantization

Tile quantization occurs when problem dimensions are not divisible by the tile size used by the kernel. For matrix multiplication, if a kernel computes output tiles of size $T_M \times T_N$, then the number of tiles is:

$$N_{\text{tiles},M} = \left\lceil \frac{M}{T_M} \right\rceil, \quad N_{\text{tiles},N} = \left\lceil \frac{N}{T_N} \right\rceil.$$

The total number of output tiles is:

$$N_{\text{tiles}} = \left\lceil \frac{M}{T_M} \right\rceil \left\lceil \frac{N}{T_N} \right\rceil.$$

If M or N is not divisible by the tile size, boundary tiles perform less useful work but still occupy hardware resources. This reduces efficiency.

3.3.2 Wave Quantization

Wave quantization occurs when the total number of tiles is not divisible by the number of SMs. If a GPU has S SMs and a kernel launches N_{tiles} independent blocks, then the number of full waves is approximately:

$$N_{\text{waves}} = \left\lceil \frac{N_{\text{tiles}}}{S} \right\rceil.$$

If the final wave contains only a few blocks, many SMs sit idle. This can reduce achieved occupancy and make kernel duration worse.

Tile quantization wastes work inside blocks. Wave quantization wastes parallelism across SMs.

3.4 Using the CUDA Occupancy Calculator API

CUDA provides an API to suggest a reasonable block size and minimum grid size:

```
int min_grid_size = 0;
int block_size = 0;

cudaOccupancyMaxPotentialBlockSize(
    &min_grid_size,
    &block_size,
    copy_data_coalesced,
    0,
    0
);

printf("Recommended block size: %d\n", block_size);
printf("Minimum grid size: %d\n", min_grid_size);
```

The function uses hardware-specific limits and kernel resource usage to recommend launch parameters. The lecture emphasizes that the answer can vary across GPUs. A T4 and an A100 may produce different recommended block and grid sizes for the same kernel.

This API is not a proof of optimality. It is a strong first heuristic. The correct workflow is still to benchmark and profile.

3.5 Memory-Bound Versus Compute-Bound Kernels

A central diagnostic question is:

Is the kernel limited by moving data or by doing arithmetic?

If a kernel is memory-bound, the hardware spends much of its time waiting for data. Optimizations should reduce memory traffic or increase work per memory access.

If a kernel is compute-bound, the hardware arithmetic units are the bottleneck. Optimizations should improve the algorithm, increase tensor core utilization, improve instruction mix, or reduce unnecessary arithmetic.

3.6 Roofline Model

The roofline model relates arithmetic intensity to achieved performance.

Arithmetic intensity is:

$$I = \frac{\text{operations}}{\text{bytes moved}}.$$

Achieved performance is bounded by:

$$P \leq \min(P_{\max}, B_{\max} \cdot I),$$

where:

- P is achieved performance, usually FLOPs per second.
- $P_{\max}$ is peak compute throughput.
- $B_{\max}$ is peak memory bandwidth.
- I is arithmetic intensity.

The sloped part of the roofline is memory-bandwidth-limited:

$$P \approx B_{\max} \cdot I.$$

The flat part is compute-limited:

$$P \approx P_{\max}.$$

A kernel with low arithmetic intensity is usually memory-bound. A kernel with high arithmetic intensity may become compute-bound.

Visual Concept

Draw a log-log roofline plot. Put arithmetic intensity on the x-axis and performance on the y-axis. Draw a rising diagonal line labeled memory bandwidth limit and a horizontal line labeled peak compute. The intersection is the transition from memory-bound to compute-bound behavior.

3.7 Control Divergence

Control divergence occurs when threads in the same warp take different branches.

A warp executes one instruction stream at a time. If some threads take an `if` branch and others take the `else` branch, the warp must execute both paths with masks enabling only the relevant lanes. Threads not on the active path wait.

A divergent kernel example is:

```
__global__ void process_with_divergence(int* data, int n) {
    int index = blockIdx.x * blockDim.x + threadIdx.x;

    if (index < n) {
        if (data[index] % 2 == 0) {
            data[index] = data[index] * 2;
        } else {
            data[index] = data[index] + 1;
        }
    }
}
```

If half the warp sees even values and half sees odd values, the warp serializes the two paths. The multiplication path and addition path cannot fully run in parallel within that warp.

A branch-reduced algebraic rewrite is:

```
__global__ void process_less_divergent(int* data, int n) {
    int index = blockIdx.x * blockDim.x + threadIdx.x;

    if (index < n) {
        int value = data[index];
        int is_even = (value % 2 == 0);
        data[index] = is_even * (value * 2)
                     + (1 - is_even) * (value + 1);
    }
}
```

The expression implements:

$$y = \begin{cases} 2x, & x \text{ even,} \\ x + 1, & x \text{ odd.} \end{cases}$$

The algebraic form is:

$$y = \mathbf{1}_{\text{even}}(x) \cdot 2x + (1 - \mathbf{1}_{\text{even}}(x))(x + 1).$$

This can reduce branch instructions and make warp execution more uniform. The lecture reports a large timing improvement in the toy example, roughly from 0.74 to 0.24 in the demonstrated timing units.

3.8 Thread Coarsening

Thread coarsening means assigning more than one element of work to each thread. Instead of each thread processing one element, each thread processes a small contiguous chunk.

A standard vector addition kernel is:

```
__global__ void vector_add(float* c, const float* a, const float* b, int n) {  
    int index = blockIdx.x * blockDim.x + threadIdx.x;  
  
    if (index < n) {  
        c[index] = a[index] + b[index];  
    }  
}
```

A coarsened version with factor 2 is:

```
__global__ void vector_add_coarsened(  
    float* c,  
    const float* a,  
    const float* b,  
    int n  
) {  
    int base = 2 * (blockIdx.x * blockDim.x + threadIdx.x);  
  
    if (base < n) {  
        c[base] = a[base] + b[base];  
    }  
  
    if (base + 1 < n) {  
        c[base + 1] = a[base + 1] + b[base + 1];  
    }  
}
```

The coarsening factor is:

$$C = \frac{\text{elements processed per thread}}{\text{baseline elements processed per thread}}.$$

For the coarsened kernel above, $C = 2$.

Thread coarsening can help memory-bound kernels by:

- Reducing indexing overhead per element.
- Increasing useful work per launched thread.
- Reducing scheduling overhead.
- Improving instruction-level locality.
- Potentially improving cache behavior.

However, too much coarsening can hurt because it increases per-thread work, register pressure, and may reduce occupancy. The lecture notes that a coarsening factor of 2 produced a dramatic speedup in the live demonstration, while larger factors such as 4 or 8 did not necessarily improve much further.

3.9 Privatization

Privatization means accumulating or manipulating data in private storage before writing back to global memory. Private storage may mean registers, local variables, or shared memory.

A simple register-level privatization pattern is:

```
__global__ void add_private(float* c, const float* a, const float* b, int n) {
    int index = blockIdx.x * blockDim.x + threadIdx.x;

    if (index < n) {
        float a_private = a[index];
        float b_private = b[index];
        float result = a_private + b_private;
        c[index] = result;
    }
}
```

This trivial version may not improve performance because the compiler already performs similar register allocation. The more important privatization pattern occurs when the same data is reused many times.

For a sliding-window sum, a naive approach repeatedly reads global memory:

```
__global__ void sliding_sum_global(
    float* out,
    const float* input,
    int n,
    int radius
) {
    int index = blockIdx.x * blockDim.x + threadIdx.x;

    if (index < n) {
        float sum = 0.0f;

        for (int offset = -radius; offset <= radius; ++offset) {
            int j = index + offset;

            if (0 <= j && j < n) {
                sum += input[j];
            }
        }

        out[index] = sum;
    }
}
```

A shared-memory version stages a block of data into faster on-chip memory:

```
__global__ void sliding_sum_shared(
    float* out,
    const float* input,
    int n,
    int radius
) {
    extern __shared__ float tile[];
```

```

int local = threadIdx.x;
int global = blockIdx.x * blockDim.x + threadIdx.x;

if (global < n) {
    tile[local] = input[global];
} else {
    tile[local] = 0.0f;
}

__syncthreads();

if (global < n) {
    float sum = 0.0f;

    for (int offset = -radius; offset <= radius; ++offset) {
        int neighbor = local + offset;

        if (0 <= neighbor && neighbor < blockDim.x) {
            sum += tile[neighbor];
        } else {
            int global_neighbor = global + offset;

            if (0 <= global_neighbor && global_neighbor < n) {
                sum += input[global_neighbor];
            }
        }
    }

    out[global] = sum;
}
}

```

The key idea is:

Read once from global memory → reuse many times from shared memory → write once to global memory

3.10 Tiling Reused Data

Tiling is a structured form of privatization. It is especially important for matrix multiplication and attention.

For matrix multiplication:

$$C = AB,$$

where:

$$A \in \mathbb{R}^{M \times N}, \quad B \in \mathbb{R}^{N \times K}, \quad C \in \mathbb{R}^{M \times K}.$$

The output element is:

$$C_{i,j} = \sum_{\ell=0}^{N-1} A_{i,\ell} B_{\ell,j}.$$

A naive kernel repeatedly loads values of A and B from global memory. A tiled kernel loads submatrices of A and B into shared memory, reuses them for many multiply-add operations, and then moves to the next tile.

A simplified tiled CUDA structure is:

```
#define TILE 16

__global__ void matmul_tiled(
    float* C,
    const float* A,
    const float* B,
    int M,
    int N,
    int K
) {
    __shared__ float As[TILE][TILE];
    __shared__ float Bs[TILE][TILE];

    int row = blockIdx.y * TILE + threadIdx.y;
    int col = blockIdx.x * TILE + threadIdx.x;

    float acc = 0.0f;

    for (int tile = 0; tile < (N + TILE - 1) / TILE; ++tile) {
        int a_col = tile * TILE + threadIdx.x;
        int b_row = tile * TILE + threadIdx.y;

        if (row < M && a_col < N) {
            As[threadIdx.y][threadIdx.x] = A[row * N + a_col];
        } else {
            As[threadIdx.y][threadIdx.x] = 0.0f;
        }

        if (b_row < N && col < K) {
            Bs[threadIdx.y][threadIdx.x] = B[b_row * K + col];
        } else {
            Bs[threadIdx.y][threadIdx.x] = 0.0f;
        }

        __syncthreads();

        for (int inner = 0; inner < TILE; ++inner) {
            acc += As[threadIdx.y][inner] * Bs[inner][threadIdx.x];
        }

        __syncthreads();
    }

    if (row < M && col < K) {
        C[row * K + col] = acc;
    }
}
```

The synchronization calls are necessary because all threads in the block collaboratively load

shared memory. A thread must not consume a tile before its neighbors have finished loading it.

4 Hardware-Level Explanation

4.1 Why DRAM Is Expensive

Global memory has high latency because it is physically farther from the SMs and implemented as DRAM. Even when bandwidth is enormous, latency remains difficult to reduce.

A simplified latency hierarchy is:

$$\text{registers} < \text{shared memory} \approx \text{L1 cache} < \text{L2 cache} < \text{global memory}.$$

The lecture refers to microbenchmarking studies that estimate latencies such as:

$$\text{global memory} \approx 290 \text{ cycles}, \quad \text{L2 cache} \approx 200 \text{ cycles}, \quad \text{L1/shared memory} \ll \text{global memory}.$$

The exact numbers vary by architecture and benchmark methodology. The important point is not the precise cycle count but the order of magnitude: global memory is much slower than on-chip memory.

4.2 Why Coalescing Helps Hardware

Memory systems operate on cache lines or memory transaction segments, not individual scalar values. If 32 threads in a warp read 32 adjacent `float` values, the hardware can serve them with a small number of wide memory transactions.

For `float`, each element has 4 bytes. A warp reading 32 contiguous floats requests:

$$32 \cdot 4 = 128 \text{ bytes}.$$

That maps naturally onto common memory transaction sizes. If the same warp reads scattered addresses, the hardware may need many transactions:

$$\text{transactions}_{\text{scattered}} > \text{transactions}_{\text{coalesced}}.$$

More transactions mean:

- More DRAM traffic.
- More cache pressure.
- More memory latency exposure.
- Lower effective bandwidth.

4.3 Warp Scheduling and Latency Hiding

An SM contains warp schedulers. A warp scheduler selects ready warps and issues instructions. If a warp stalls on memory, the scheduler can switch to another warp.

Latency hiding requires enough independent ready warps:

$$\text{latency hidden} \iff \text{many independent warps are resident and ready}.$$

Low occupancy means fewer warps are available to cover stalls. However, high occupancy alone is insufficient if all warps stall on the same bottleneck.

4.4 Register Pressure and Shared Memory Pressure

Occupancy is limited by per-block resource usage. If each thread uses R registers and each block has T threads, then registers per block are:

$$R_{\text{block}} = R \cdot T.$$

If an SM has R_{SM} registers, then the maximum number of resident blocks due to registers is approximately:

$$B_{\text{reg}} = \left\lfloor \frac{R_{\text{SM}}}{R \cdot T} \right\rfloor.$$

Similarly, if each block uses S_{block} bytes of shared memory and the SM has S_{SM} bytes available, the shared-memory limit is:

$$B_{\text{smem}} = \left\lfloor \frac{S_{\text{SM}}}{S_{\text{block}}} \right\rfloor.$$

The actual resident block count is bounded by the minimum of all resource limits:

$$B_{\text{resident}} = \min(B_{\text{reg}}, B_{\text{smem}}, B_{\text{thread}}, B_{\text{warp}}, B_{\text{arch}}).$$

This is why optimizations such as tiling and coarsening can have trade-offs. They may reduce memory traffic but increase registers or shared memory enough to reduce occupancy.

4.5 Control Divergence at Warp Level

For a branch:

if condition then A else B,

if all 32 threads in a warp agree on the condition, only one path executes. If some threads take A and some take B , the warp executes both paths serially with masks.

Let p_A be the fraction of lanes taking path A , and p_B be the fraction taking path B . If both are nonzero, the warp may pay approximately:

$$T_{\text{divergent}} \approx T_A + T_B,$$

instead of:

$$T_{\text{uniform}} \approx \max(T_A, T_B)$$

for perfectly separated work across different warps. The precise hardware behavior is more complex, but the mental model is that branch disagreement inside a warp serializes paths.

5 Mathematical Reasoning

5.1 Arithmetic Intensity of ReLU

Consider ReLU applied elementwise:

$$y_i = \max(0, x_i).$$

For each element, the kernel performs approximately one comparison. If $x_i \leq 0$, it may write 0. If $x_i > 0$, it may write x_i , depending on whether the operation is out-of-place.

For an out-of-place FP32 ReLU, each element requires:

$$\text{bytes moved} = \underbrace{4}_{\text{read } x_i} + \underbrace{4}_{\text{write } y_i} = 8.$$

The operation count is approximately:

$$\text{ops} = 1.$$

Thus arithmetic intensity is:

$$I_{\text{ReLU, FP32}} = \frac{1}{8} = 0.125 \quad \text{ops per byte.}$$

For FP16, each element is 2 bytes:

$$\text{bytes moved} = 2 + 2 = 4,$$

so:

$$I_{\text{ReLU, FP16}} = \frac{1}{4} = 0.25 \quad \text{ops per byte.}$$

This explains why quantization or lower precision can improve arithmetic intensity. The amount of arithmetic is similar, but the number of bytes moved decreases.

5.2 Arithmetic Intensity of Matrix Multiplication

Let:

$$A \in \mathbb{R}^{M \times N}, \quad B \in \mathbb{R}^{N \times K}, \quad C \in \mathbb{R}^{M \times K}.$$

Each output element is:

$$C_{i,j} = \sum_{\ell=0}^{N-1} A_{i,\ell} B_{\ell,j}.$$

For each $C_{i,j}$, there are approximately N multiplications and N additions:

$$\text{FLOPs per output element} \approx 2N.$$

There are MK output elements, so:

$$\text{FLOPs} \approx 2MNK.$$

Ignoring cache reuse, the memory footprint in elements is:

$$MN + NK + MK.$$

For FP32, the byte movement is approximately:

$$4(MN + NK + MK).$$

Thus arithmetic intensity is:

$$I_{\text{GEMM, FP32}} = \frac{2MNK}{4(MN + NK + MK)} = \frac{MNK}{2(MN + NK + MK)}.$$

If we count in elements rather than bytes, the lecture's simplified expression is:

$$I_{\text{GEMM, elements}} = \frac{2MNK}{MN + NK + MK}.$$

For square matrices where $M = N = K = n$:

$$I_{\text{GEMM, elements}} = \frac{2n^3}{3n^2} = \frac{2n}{3}.$$

Thus, as n grows, matrix multiplication becomes increasingly compute-intensive. This is why GEMM is usually compute-bound for large matrices, while elementwise operations are often memory-bound.

5.3 Padding and Tensor Core Alignment

The lecture discusses why dimensions that are multiples of certain values often perform better. Tensor cores and memory systems prefer aligned dimensions. The preferred multiple depends on datatype and architecture.

A general alignment rule is:

$$D \equiv 0 \pmod{A},$$

where:

- D is a matrix dimension such as M , N , or K .
- A is the architecture-specific alignment multiple.

For example, lower-precision tensor core paths often prefer larger alignment multiples than FP32 paths. This is why deep learning model dimensions are frequently powers of two or multiples of 8, 16, 32, 64, or 128.

Padding changes the effective dimension:

$$D_{\text{padded}} = A \left\lceil \frac{D}{A} \right\rceil.$$

The overhead is:

$$\text{padding overhead} = \frac{D_{\text{padded}} - D}{D}.$$

Padding can increase arithmetic work but improve hardware utilization enough to reduce wall-clock time.

5.4 Softmax and Online Normalization

The standard softmax is:

$$y_i = \frac{e^{x_i}}{\sum_{j=1}^n e^{x_j}}.$$

The denominator is:

$$d = \sum_{j=1}^n e^{x_j}.$$

A naive implementation needs one pass to compute d , then another pass to compute each y_i . The numerically safe softmax subtracts the maximum:

$$m = \max_{1 \leq j \leq n} x_j,$$
$$y_i = \frac{e^{x_i - m}}{\sum_{j=1}^n e^{x_j - m}}.$$

This avoids overflow because $x_i - m \leq 0$, so:

$$e^{x_i - m} \leq 1.$$

However, safe softmax usually needs:

1. One pass to compute m .
2. One pass to compute the denominator.
3. One pass to compute and store the normalized outputs.

Online softmax combines the maximum and denominator updates into one streaming recurrence. Let m_j be the maximum after seeing elements 1 through j :

$$m_j = \max(m_{j-1}, x_j).$$

Let d_j be the corrected denominator after seeing elements 1 through j :

$$d_j = d_{j-1} e^{m_{j-1} - m_j} + e^{x_j - m_j}.$$

This recurrence works because if the maximum changes, the old partial denominator must be rescaled to the new maximum. The exponential identity is:

$$e^{x - m_{\text{old}}} = e^{x - m_{\text{new}}} e^{m_{\text{new}} - m_{\text{old}}},$$

or equivalently the old sum can be corrected by:

$$d_{\text{old corrected}} = d_{\text{old}} e^{m_{\text{old}} - m_{\text{new}}}.$$

If the maximum does not change, then:

$$m_j = m_{j-1},$$

so:

$$e^{m_{j-1}-m_j} = e^0 = 1,$$

and the recurrence reduces to:

$$d_j = d_{j-1} + e^{x_j - m_j}.$$

Thus online softmax computes the same denominator as safe softmax but with fewer global memory passes.

5.5 Connection to FlashAttention

Scaled dot-product attention is:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right) V.$$

The difficulty is the softmax. Matrix multiplication can be tiled naturally, but softmax requires a row-wise global denominator:

$$d_i = \sum_j \exp\left(\frac{Q_i \cdot K_j}{\sqrt{d}} - m_i\right).$$

FlashAttention uses online softmax statistics to process blocks of K and V while maintaining the correct row-wise maximum and denominator. This allows attention to be tiled through SRAM without materializing the full attention matrix in global memory.

The important state per query row is:

m_i = running row maximum, ℓ_i = running row denominator, o_i = running output accumulator.

When a new block arrives, the algorithm updates m_i , rescales the old denominator and output accumulator, incorporates the new block, and continues.

6 Code Walkthrough

6.1 Benchmark Harness Structure

A typical benchmark allocates memory, initializes launch parameters, launches kernels, synchronizes, and frees memory.

```
int main() {
    int n = 1 << 24;

    float* input;
    float* output;

    cudaMallocManaged(&input, n * sizeof(float));
    cudaMallocManaged(&output, n * sizeof(float));

    int block_size = 128;
    int num_blocks = (n + block_size - 1) / block_size;
```

```

copy_data_coalesced<<<num_blocks, block_size>>>(output, input, n);
cudaDeviceSynchronize();

cudaFree(input);
cudaFree(output);

return 0;
}

```

The grid-size formula:

$$\text{num_blocks} = \left\lceil \frac{n}{\text{block_size}} \right\rceil = \frac{n + \text{block_size} - 1}{\text{block_size}}$$

in integer arithmetic ensures enough blocks are launched to cover all n elements.

6.2 Coalesced Copy

```

__global__ void copy_data_coalesced(float* out, const float* in, int n) {
    int index = blockIdx.x * blockDim.x + threadIdx.x;

    if (index < n) {
        out[index] = in[index];
    }
}

```

Line-by-line hardware behavior:

- The index calculation maps each CUDA thread to one global element.
- The bounds check prevents out-of-range accesses in the final partial block.
- Adjacent threads read adjacent float values.
- Adjacent threads write adjacent float values.
- Memory access is coalesced, improving effective bandwidth.

The kernel is memory-bound because it performs almost no arithmetic:

$$I \approx \frac{0}{\text{bytes moved}} \approx 0.$$

6.3 Non-Coalesced Copy

```

__global__ void copy_data_noncoalesced(float* out, const float* in, int n) {
    int index = blockIdx.x * blockDim.x + threadIdx.x;

    if (index < n) {
        int read_index = (2 * index) % n;
        out[index] = in[read_index];
    }
}

```

This kernel has similar semantic work but worse memory behavior.
Major issues:

- The read pattern skips elements.
- Threads in a warp no longer map cleanly to one contiguous memory segment.
- L1 cache throughput can be worse.
- DRAM throughput can increase.
- Kernel duration increases even though the arithmetic is almost unchanged.

6.4 Occupancy-Guided Launch Configuration

```
int min_grid_size = 0;
int block_size = 0;

cudaOccupancyMaxPotentialBlockSize(
    &min_grid_size,
    &block_size,
    copy_data_coalesced,
    0,
    0
);

int grid_size = (n + block_size - 1) / block_size;

copy_data_coalesced<<<grid_size, block_size>>>(output, input, n);
cudaDeviceSynchronize();
```

The recommended block size is hardware-dependent. A larger GPU does not necessarily imply a larger optimal block size. The API accounts for architectural limits and kernel resource usage.

6.5 Branch-Divergent Kernel

```
__global__ void process_with_divergence(int* data, int n) {
    int index = blockIdx.x * blockDim.x + threadIdx.x;

    if (index < n) {
        if (data[index] % 2 == 0) {
            data[index] = data[index] * 2;
        } else {
            data[index] = data[index] + 1;
        }
    }
}
```

The problem is not the existence of `if` statements alone. The problem is data-dependent disagreement inside a warp. If neighboring threads see different parity values, the warp executes both paths.

6.6 Branch-Reduced Kernel

```
__global__ void process_less_divergent(int* data, int n) {
    int index = blockIdx.x * blockDim.x + threadIdx.x;

    if (index < n) {
        int value = data[index];
        int is_even = (value % 2 == 0);

        data[index] = is_even * (value * 2)
                     + (1 - is_even) * (value + 1);
    }
}
```

This removes the inner branch by computing both candidate expressions and selecting algebraically. This can be beneficial when branch divergence costs more than the extra arithmetic.

The trade-off is:

$$\text{branch elimination benefit} > \text{extra arithmetic cost.}$$

This is often true for simple arithmetic and highly divergent branches, but not always true for expensive branch bodies.

6.7 Thread-Coarsened Vector Addition

```
__global__ void vector_add_coarsened(
    float* c,
    const float* a,
    const float* b,
    int n
) {
    int thread_index = blockIdx.x * blockDim.x + threadIdx.x;
    int base = 2 * thread_index;

    if (base < n) {
        c[base] = a[base] + b[base];
    }

    if (base + 1 < n) {
        c[base + 1] = a[base + 1] + b[base + 1];
    }
}
```

The launch grid must account for the coarsening factor:

$$\text{num_blocks} = \left\lceil \frac{n}{C \cdot \text{block_size}} \right\rceil.$$

For $C = 2$:

$$\text{num_blocks} = \left\lceil \frac{n}{2 \cdot \text{block_size}} \right\rceil.$$

This reduces the number of threads needed to cover the same data. The kernel remains memory-bound, but it may reduce scheduling and indexing overhead.

7 Performance Intuition

7.1 Checklist for Memory-Bound Kernels

A kernel is likely memory-bound when:

- Arithmetic intensity is low.
- Nsight Compute reports memory more heavily utilized than compute.
- DRAM throughput is high.
- The kernel performs simple elementwise operations.
- Runtime scales with bytes moved more than with operations.

Useful optimizations include:

- Coalesce global memory accesses.
- Fuse multiple elementwise kernels.
- Use thread coarsening.
- Use lower precision or quantization.
- Recompute cheap values instead of loading them.
- Use shared memory or registers for reuse.
- Avoid unnecessary writes.

7.2 Checklist for Compute-Bound Kernels

A kernel is likely compute-bound when:

- Arithmetic intensity is high.
- Compute pipelines are highly utilized.
- Memory throughput is not the limiting metric.
- The operation resembles large GEMM, convolution, or attention.

Useful optimizations include:

- Improve tiling.
- Increase tensor core utilization.
- Use better instruction shapes.
- Reduce redundant arithmetic.
- Improve data layout for tensor core compatibility.
- Use specialized libraries such as cuBLAS, cuDNN, CUTLASS, or custom Triton/CUDA kernels.
- Rewrite the algorithm if the mathematical structure allows it.

7.3 Why Fusion Helps Memory-Bound Workloads

Suppose two elementwise kernels are executed separately:

$$z_i = f(x_i), \quad y_i = g(z_i).$$

Separate kernels require:

$$\text{read } x \rightarrow \text{write } z \rightarrow \text{read } z \rightarrow \text{write } y.$$

A fused kernel computes:

$$y_i = g(f(x_i)).$$

Now the memory traffic is:

$$\text{read } x \rightarrow \text{write } y.$$

The intermediate z_i can live in a register:

$$z_i \in \text{registers}.$$

Fusion reduces global memory traffic and kernel launch overhead.

7.4 Why Quantization Helps Memory-Bound Workloads

If an FP32 tensor has n elements, its memory footprint is:

$$4n \text{ bytes.}$$

For FP16:

$$2n \text{ bytes.}$$

For INT8:

$$n \text{ bytes.}$$

For INT4:

$$\frac{n}{2} \text{ bytes.}$$

Reducing bytes moved increases arithmetic intensity:

$$I = \frac{\text{ops}}{\text{bytes moved}}.$$

However, quantization can introduce overhead for dequantization and can reduce numerical quality. There is no free lunch: it improves memory movement but may affect accuracy or add conversion operations.

7.5 Why Algorithmic Math Tricks Matter

Some optimizations cannot be discovered safely by a compiler because they require changing the mathematical form of the program while preserving semantics. Online softmax is a prime example.

The original safe softmax reads the data multiple times. Online softmax changes the recurrence so the denominator and maximum can be updated together. This reduces memory traffic while preserving numerical correctness.

The strongest GPU programmers combine mathematics and systems knowledge. They know how to change the algorithm so that it exposes the right hardware behavior.

8 Common Misconceptions

8.1 Misconception: Global Memory Bandwidth Means Global Memory Is Cheap

High bandwidth does not mean low latency. GPUs hide memory latency using parallelism, but individual memory accesses are still expensive.

8.2 Misconception: Always Maximize Occupancy

Occupancy is important, but maximum occupancy is not always maximum performance. A kernel with lower occupancy but much better data reuse can be faster.

8.3 Misconception: More Threads Always Means Faster

More threads help only if they expose useful parallelism and hide latency. Too many threads can increase scheduling overhead, register pressure, and shared memory pressure.

8.4 Misconception: Powers of Two Are Magic

Powers of two are not magic by themselves. They often work well because hardware instructions, memory alignment, and tensor core tile sizes prefer certain multiples.

8.5 Misconception: Branches Are Always Bad

Branches are bad when they cause warp divergence. If every thread in a warp takes the same branch, the cost is much lower.

8.6 Misconception: Coarsening Always Helps

Thread coarsening can improve memory-bound kernels, but excessive coarsening can reduce occupancy and increase register pressure.

8.7 Misconception: Shared Memory Automatically Makes Code Faster

Shared memory helps only when data is reused enough to justify the cost of loading it, synchronizing, and managing layout. For one-time reads, shared memory may not help.

8.8 Misconception: Compilers Can Find Every Optimization

Compilers can fuse, simplify, and optimize many patterns, but they usually cannot invent a new mathematical algorithm such as online softmax or FlashAttention.

9 Practical Takeaways

1. Start with profiling, not guessing.
2. Inspect kernel duration first; other metrics matter only insofar as they explain runtime.
3. For memory-bound kernels, reduce bytes moved.
4. For compute-bound kernels, improve arithmetic efficiency and algorithmic structure.
5. Make global memory accesses contiguous across adjacent threads.
6. Use occupancy tools as heuristics, then validate with real benchmarks.
7. Watch for tile and wave quantization when dimensions are awkward.
8. Avoid data-dependent branch divergence inside warps.
9. Use thread coarsening for simple memory-bound kernels, but benchmark multiple coarsening factors.
10. Use shared memory when data reuse is high.
11. Prefer registers for private scalar temporaries.
12. Pad dimensions when it improves tensor core or memory alignment.
13. Learn the mathematical identities behind high-performance kernels.

10 Project or Experiment Ideas

10.1 Experiment 1: Coalesced Versus Strided Copy

Implement two copy kernels:

$$\text{out}[i] = \text{in}[i]$$

and

$$\text{out}[i] = \text{in}[(s \cdot i) \bmod n] \quad \text{out}[i] = \text{in}[(s \cdot i) \bmod n] \quad \text{out}[i] = \text{in}[(s \cdot i) \bmod n] \quad \text{out}[i] = \text{in}[(s \cdot i) \bmod n].$$

Vary stride s over:

$$s \in \{1, 2, 4, 8, 16, 32\}.$$

Measure:

- Kernel duration.

- DRAM throughput.
- L1 cache throughput.
- Global load efficiency.

10.2 Experiment 2: Occupancy Calculator Versus Manual Block Sizes

Run the same memory copy or vector add kernel with block sizes:

32, 64, 128, 256, 512, 1024.

Compare against `cudaOccupancyMaxPotentialBlockSize`. Record which block size is actually fastest.

10.3 Experiment 3: Thread Coarsening Sweep

Implement vector addition with coarsening factors:

$C \in \{1, 2, 4, 8, 16\}$.

Measure runtime and register usage. Identify where coarsening stops helping.

10.4 Experiment 4: Divergence Stress Test

Create kernels with increasingly divergent branches. Use input arrays with:

- All even values.
- All odd values.
- Alternating even and odd values.
- Random parity.

Compare branch counts, warp execution efficiency, and kernel runtime.

10.5 Experiment 5: Naive Versus Safe Versus Online Softmax

Implement:

1. Naive softmax.
2. Safe softmax with max subtraction.
3. Online softmax.

Measure:

- Number of memory passes.
- Runtime.
- Numerical stability for large inputs.
- Maximum absolute error compared with PyTorch.

10.6 Experiment 6: Tiled Matrix Multiplication

Implement naive and tiled matrix multiplication. Vary tile sizes:

$$T \in \{8, 16, 32\}.$$

Measure:

- Runtime.
- Shared memory usage.
- Occupancy.
- Global memory throughput.
- Achieved FLOPs.

10.7 Experiment 7: Padding for GEMM

Benchmark matrix multiplications where K varies from 1000 to 1024. Compare unpadded and padded dimensions. Observe how alignment affects performance.

11 Final Mental Model

CUDA performance is the art of matching the algorithm to the GPU memory hierarchy and execution model. A GPU has enormous arithmetic throughput, but that throughput is accessible only when the kernel feeds the hardware efficiently, keeps warps busy, avoids unnecessary global memory traffic, and minimizes divergence.

The checklist can be compressed into one decision process:

Profile $\rightarrow$ identify bottleneck $\rightarrow$ choose targeted optimization $\rightarrow$ re-profile.

For memory-bound kernels:

reduce bytes moved or do more work per byte.

For compute-bound kernels:

improve the algorithm or use the hardware's preferred compute path.

For warp-level inefficiency:

make neighboring threads do similar work on neighboring data.

For high-performance deep learning kernels, the best solutions usually combine all of these ideas. FlashAttention is the canonical example: it uses tiling, SRAM reuse, online softmax, reduced memory traffic, and mathematical reformulation to turn a memory-heavy attention algorithm into a much more hardware-efficient kernel.

The final mental model is simple: global memory is expensive, warps want uniform work, SMs need enough independent work to hide latency, and the best CUDA kernels are designed from the math down to the memory transactions.